

Fundamentals of Programming: Python Basics

Prepared by the teaching team for the Fundamentals of Programming course at the University of Science and Technology.

This lecture covers the core building blocks of Python programming, aimed at complete beginners. We will explore **Input and Output**, **Variables**, **Keywords**, **Operators**, **Booleans**, **Data Types**, **Numbers**, **Casting**, and **Comments** in Python. Each concept is explained in detail with clear examples and code. By the end of this lecture, you will have a solid understanding of basic Python syntax and semantics.

Input and Output

Python provides built-in functions to interact with the user. The `print()` function sends output to the console, while the `input()` function reads user input ¹ ². For example, `print("Hello")` will display *Hello* on the screen, and `input("Enter your name: ")` will prompt the user to type something and then return it as a string.

- **Printing Output:** The `print()` function prints the specified message to the standard output (usually the screen) ¹. It can print strings, numbers, variables, and even multiple items separated by commas (with a space in between by default).
- **Reading Input:** The `input()` function allows user input ². By default, it returns the user's input as a string. You can optionally provide a prompt message inside the parentheses.

```
# Example: Using print() and input() together
name = input("Enter your name: ")    # Prompts user and reads input as string
print("Hello,", name, "! Welcome!")  # Greets the user
```

```
# Example: Converting input to a number
age = int(input("Enter your age: ")) # input() reads string, int() converts to integer
print("In 5 years you will be", age + 5)
```

- **Formatted Output:** You can format output using f-strings or the `format()` method. For example:

```
x = 5; y = 10
print(f"x = {x}, y = {y}")          # f-string formatting
print("x = {}, y = {}".format(x, y)) # format() method
```

These simple I/O functions let your Python programs interact with users through the console.

Variables

A **variable** in Python is a name that refers to a value stored in memory. Variables allow us to store and manipulate data. In Python, you create a variable by assigning it a value with the `=` operator ³. Unlike some languages, Python variables do **not** require an explicit declaration of type; they are dynamically typed, meaning the type is inferred from the assigned value ³.

For example:

```
x = 10          # x now refers to the integer 10
message = "Hello, World!" # message is a string
pi = 3.14       # pi is a float
```

You can reassign a variable to a value of a different type:

```
x = 5          # x is int
x = "five"     # now x is a string
```

Variable Naming Rules: Variable names must start with a letter or underscore, followed by letters, digits, or underscores. They are case-sensitive (e.g., `data` and `Data` are different) ⁴. For example: - Valid names: `count`, `_temp`, `number1`, `user_name`
- Invalid names: `1stValue` (starts with digit), `for` (reserved keyword) ⁴.

Multiple Assignment: Python allows assigning values to multiple variables in one line. For example:

```
a = b = c = 0          # assigns 0 to all three variables
x, y, name = 1, 2.5, "Python" # assigns 1 to x, 2.5 to y, "Python" to name
```

This is a convenient way to initialize variables or swap values. For instance, swapping two variables without a temporary is easy with tuple unpacking:

```
a, b = 5, 10
a, b = b, a
print(a, b) # Outputs: 10 5
```

Variables are references to objects, and reassigning a variable changes which object it refers to ⁵. You can also delete a variable using `del`, after which it no longer exists.

Keywords

Python has a set of **reserved keywords** that have special meanings and cannot be used as variable names or identifiers ⁶. Keywords define the syntax of the language (e.g., control structures, data types). For example, you cannot name a variable `if`, `else`, `for`, `while`, `class`, or `return`, etc. ⁶. Here are some common keywords and their roles:

- `if`, `else`, `elif` – Used in conditional statements.
- `for`, `while` – Used for loops.
- `def`, `return` – Define and return from functions.
- `class` – Define a class.
- `import`, `from`, `as` – Import modules and rename them.
- `and`, `or`, `not`, `in`, `is` – Logical and membership operators (also reserved words).
- `True`, `False`, `None` – Boolean values and null (also keywords).

Important: Since keywords are reserved, trying to use one as a variable name will result in a syntax error ⁶. Always choose meaningful names that do not clash with Python's keywords.

Operators

In Python, **operators** are special symbols that perform operations on values or variables ⁷. They include arithmetic operators, comparison operators, logical operators, and more ⁷ ⁸. Here are the main types:

- **Arithmetic Operators:** For mathematical calculations. These include `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `//` (floor division), `%` (modulus), and `**` (exponentiation) ⁹. Example:

```
print(10 + 5)    # 15
print(7 * 3)     # 21
print(2 ** 4)    # 16 (2 to the power of 4)
print(7 // 3)    # 2 (integer division)
print(7 % 3)     # 1 (remainder)
```

- **Comparison (Relational) Operators:** Compare two values and return a Boolean (`True` or `False`) ¹⁰. These include `==` (equal), `!=` (not equal), `>`, `<`, `>=`, `<=`. Example:

```
print(10 > 9)    # True
print(5 == 5)    # True
print(3 != 4)    # True
print(2 <= 1)    # False
```

These are often used in `if` statements and loops.

- **Logical Operators:** Combine Boolean expressions ¹¹. The main ones are `and`, `or`, and `not`. They evaluate expressions logically. Example:

```
a = True; b = False
print(a and b)  # False (both must be True to result True)
print(a or b)   # True  (either can be True)
print(not a)    # False (negation)
```

- **Assignment Operators:** Assign values to variables. The basic one is `=`. There are also compound operators that combine assignment with arithmetic, e.g., `+=`, `-=`, `*=`, `/=` etc. For example, `x += 5` is shorthand for `x = x + 5`.

- **Identity Operators:** `is` and `is not` test whether two variables refer to the same object in memory ¹². For example:

```
x = [1, 2, 3]
y = x
z = [1, 2, 3]
print(x is y)      # True, y references same object as x
print(x is z)      # False, z is a different list with same contents
```

- **Membership Operators:** `in` and `not in` check if a value is (or is not) in a sequence (like a list or string) ¹³. Example:

```
fruits = ["apple", "banana", "cherry"]
print("banana" in fruits)  # True
print("kiwi" not in fruits) # True
```

- **Bitwise Operators:** Perform bit-level operations on integers (`&`, `|`, `^`, `~`, `<<`, `>>`). These are more advanced and not covered in detail here.

Note: Operators follow a precedence order (e.g. `*` before `+`), but parentheses `()` can override order. Use operators to build expressions; the result may be a number, string, or Boolean depending on the operators used.

Booleans

The Boolean type (`bool`) in Python represents truth values. A Boolean variable can only be **True** or **False** ¹⁴. These values are also Python keywords (`True`, `False`). Internally, `True` is treated as the integer `1` and `False` as `0`, but you almost always use them in logical contexts. For example:

```

a = True
b = False
print(type(a))      # <class 'bool'>
print(a and not b)  # True (since a is True and b is False)

```

Every comparison operator returns a Boolean. For example, the expression `1 <= 2` evaluates to `True`, while `0 == 1` evaluates to `False` ¹⁵.

You can convert other values to Boolean using the `bool()` function ¹⁶. Most values in Python are *truthy* except for a few: zero of any numeric type, empty sequences (like `""`, `()`, `[]`), and the special value `None` are all considered *False*. For instance:

```

print(bool("Hello")) # True (non-empty string)
print(bool(0))       # False (zero is False)
print(bool([]))      # False (empty list)
print(bool(None))    # False (None is like null)

```

Boolean values are essential for flow control (e.g., `if` statements) and logical operations ¹⁴ ¹⁶.

Data Types

A **data type** specifies the kind of value a variable can hold and what operations are allowed on it ¹⁷. In Python, everything is an object and each object has a data type. Common built-in data types include ¹⁸:

- **Numeric Types:**

- `int` – integer numbers, e.g. `42`, `-3` ¹⁹. Integers can be arbitrarily large (no fixed limit) ¹⁹.
- `float` – floating-point (decimal) numbers, e.g. `3.14`, `-0.001` ²⁰. Floats can also be written in scientific notation (`1e3` for 1000) ²¹.
- `complex` – complex numbers with real and imaginary parts, e.g. `2+3j` ²².

- **Boolean Type:**

- `bool` – truth values `True` or `False` ¹⁴ (subclass of numeric).

- **Sequence Types:**

- `str` – string of characters, e.g. `"hello"` (sequence of Unicode characters).
- `list` – an ordered, mutable collection of items, e.g. `[1, 2, 3]`.
- `tuple` – an ordered, immutable collection, e.g. `("a", "b")`.

- **Mapping Type:**

- `NoneType` – the type of the `None` value (represents the absence of a value).

- **Complex** (`complex`): Numbers with real and imaginary parts, using `j` to denote the imaginary unit ²². For example, `3+5j` is a complex number with real part 3 and imaginary part 5.

```
z = 2 + 3j
print(type(z)) # <class 'complex'>
```

You can convert between numeric types using functions like `int()`, `float()`, and `complex()`. For example, `int(5.7)` yields `5`, and `float(10)` yields `10.0` ²⁴ (we'll cover conversions more in *Casting*).

Basic arithmetic with numbers uses the operators mentioned earlier (e.g. `+`, `-`, `*`, `/`, `//`, `%`, `**`). Python follows standard mathematical rules (with `**` for exponents and `//` for integer division) ⁹ ²⁵.

Casting

Casting (or type conversion) is the process of converting a value from one data type to another ⁵. Python provides built-in functions for this:

- `int(x)` – converts a compatible value `x` to an integer (truncating decimals if `x` is float, or parsing if `x` is a numeric string) ²⁶.
- `float(x)` – converts `x` to a floating-point number.
- `str(x)` – converts `x` to a string representation.
- `bool(x)` – converts `x` to a Boolean (`False` if `x` is zero, empty, or `None`; otherwise `True`).

For example:

```
s = "10"
n = int(s)      # Cast string to integer 10
f = float(n)    # Cast integer 10 to float 10.0
text = str(f)   # Cast float 10.0 to string "10.0"
```

After casting, you can verify types with `type()`. Casting is often needed when reading input (since `input()` returns a string), or when performing arithmetic on mixed types. For example:

```
# Input is string by default, so convert to int for calculation
num_str = input("Enter a number: ") # e.g. user types "42"
num = int(num_str)                  # now num is integer 42
print(num + 10)                     # performs numeric addition
```

Be careful: casting can fail if the value is not compatible (e.g. `int("hello")` is invalid). Always ensure the value is of the right format before casting ⁵.

Comments

Comments are notes in the code that are not executed. They are used to explain code or disable code during testing. In Python, a **single-line comment** starts with the hash symbol `#`. Everything after `#` on that line is ignored by the interpreter ²⁷. For example:

```
# This is a single-line comment. Python will ignore this.  
print("Hello, World!") # This comment is after code on the same line
```

There is no special multi-line comment syntax in Python ²⁸. To comment out multiple lines, simply put `#` at the start of each line:

```
# This is a comment  
# spanning multiple lines.  
# Each line starts with '#'.  

```

Another common (though not official) way to add a block comment is to use a multiline string (triple quotes) by itself. A standalone string literal is ignored by Python if it is not assigned or used:

```
"""  
This is a comment spanning  
multiple lines, enclosed in  
triple quotes. Python ignores it  
if it's not assigned.  
"""  
  
print("Done")
```

However, this uses a string literal that isn't assigned to a variable. It's a handy trick for large comments or documentation strings, but the conventional way to comment is with `#`. Remember to use comments to make your code more readable and to explain non-obvious logic.

